GPU Acceleration of Recursive Fermi-operator Expansion within the Framework of a Deep Neural Network

Khadatkar Sameer Raju
Department of Computational and Data Sciences
Indian Institute of Science, Bangalore, India
sameerraju@iisc.ac.in

***Abstract*—I present a parallel GPU implementation of a second-order recursive Fermi-operator expansion scheme using mixed-precision floating-point operations to calculate electronic structure. For this, I used Nvidia's V100 GPUs with tensor core units. The paper uses a second-order recursive Fermi-operator scheme formulated in terms of a deep neural network structure, which solves the electronic structure problem of Quantum Mechanics. I demonstrate the C++ implementation of parallelizing this calculation on GPUs, evaluate its performance, and then compare it with the current state of the art.**

I. Introduction

The computation of eigenvalues and eigenvectors of a matrix forms the basis of many scientific problems. One of its applications is in Quantum mechanical calculations using Density Functional Theory, in which we need to solve a nonlinear eigenvalue problem. Solving this nonlinear eigenvalue problem is an iterative approach, and one of its intermediate steps requires constructing the single-particle electron density matrix. This density matrix is nothing but a projection operator corresponding to the lowest occupied eigenvectors of the discretized system matrix, also known as Hamiltonian. The density matrix can generally be computed by solving the eigenvalue problem associated with the Hamiltonian (H). Another popular method that allows us to construct this density matrix without explicitly solving the eigenvalue problem is the Recursive Fermi-Operator Expansion Scheme.

Density matrix recursive Fermi-operator expansion

The single-particle density matrix, D, is given by a Fermi matrix function, where

$$D = (e^{\beta(H-\mu I)} + I)^{-1} \tag{1}$$

At zero electronic temperature, this density matrix is an Heaviside step function which can be approximated by a recursive Fermi-operator expansion, where

$$D = \theta(\mu I - H) \approx f_m(f_{m-1}...(f_0(H))) \tag{2}$$

Successive passing X_i through a function f performs the recursion that starts with $X_i = H$. The $X_{i+1} = f_i(X_i)$ is calculated in each iteration until convergence is reached as $X_i \to D$. The functions $f(X)$ are chosen to converge the eigenvalue spectrum of X_i closer either to 1 or to 0. The eigenvalue is 1 for the occupied states below the chemical potential μ and 0 for the unoccupied states

1) Second-Order Spectral Projection Polynomials (SP2): In this method, the functions which are chosen in the recursive expansion functions in eq. (2) are second-order polynomials acting on the interval [0, 1], where

$$X_{i+1} = f_i(X_i) = X_i \pm (X_i - X_i^2) \tag{3}$$

The $\pm$ sign is chosen such that it adjusts the trace of X_{i+1} in each projection to the correct occupation, N_{occ} (known to us), such that $Tr[X_i] \to Tr[D] = N_{occ}$. In this way, the step is formed automatically at the correct chemical potential μ.

2) Deep NN SP2: For this formulation I refer you to section 3.2 of [1]. The structure of which is given in the fig. 1. The weights and biases, W_0, B_0, W_n, and B_n are calculated as shown in Algorithm 1, which is the sequential algorithm for this formulation.

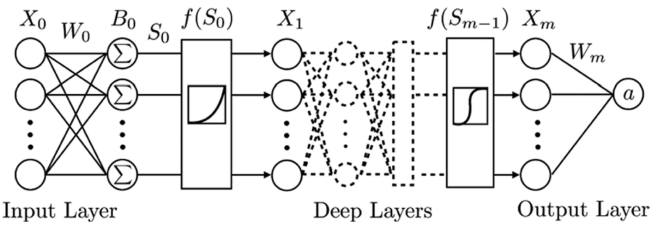


Fig. 1. Schematic picture of a deep neural network structure. The weights W_n and bias values B_n do a linear transformation of X_n, where $S_n = W_n X_n + B_n$ are given in standard matrix notation. A new layer X_{n+1} is obtained after the application of a nonlinear activation function, that is, where $X_{n+1} = f(S_n)$.

3) Mixed Precision Operation: The most expensive step in the Deep NN SP2 Fermi-operator expansion is the computation of activation function ($X_n = f(S_{n-1}) = S_{n-1}^2$). For this, we can use a mixed-precision strategy as discussed in [1]. If the precision of X is FP32, we split them into two FP16 (half precision) matrices.

$$X \approx X^{(0)} + X^{(1)} \tag{4}$$
$$X^{(0)} = FP16[X]$$

$$X^{(1)} = FP16[X - X^{(0)}] \quad (5)$$

Then if we have to do a matrix-matrix multiplication, $X \times Y$, it can be performed by using four separate matrix-matrix multiplications in half-precision and accumulated in single-precision(FP32), that is,

$$X \times Y \approx FP32[(X^{(0)} + X^{(1)})(Y^{(0)} + Y^{(1)})] \quad (6)$$

In our algorithm, we only need to calculate the square of the matrix in the activation function, and as our matrix is symmetric in each iteration, we can reduce the calculation of matrix square to only two matrix-matrix multiplication. The $X^{(1)}X^{(1)}$ term is neglected [1].

$$X^2 \approx FP32[X^{(0)}X^{(0)} + X^{(0)}X^{(1)} + (X^{(0)}X^{(1)})^T] \quad (7)$$

Algorithm 1: The Deep-NN formulation of the SP2 recursive Fermi-operator expansion algorithm

N_{occ}; Number of occupied states of orbitals
ϵ; small number close (or equal) to 0
H; Orthonormalized Hamiltonian
$\varepsilon_1, \varepsilon_N$; Spectral bound estimates of H
W_n; Observable operator matrix e.g. $W_n = H$
$X_0 = H$; Input layer
$W_0 = -(\varepsilon_N - \varepsilon_1)^{-1}$, $B_0 = \varepsilon_N(\varepsilon_N - \varepsilon_1)^{-1}I$;
$S_0 = W_0X_0 + B_0$, $N_s = Tr[S_0]$; Occupation of S_0
$n = 0$; Number of layers
while *Not Converged* **do**
  $n = n + 1$;
  $X_n = f(S_{n-1})$;
  $N_x = Tr[X_n]$;
  $IdErr_n = N_s - N_x$; Idempotency error
  $\sigma_n = Sgn(|2N_s - N_x - N_{occ}| - |N_x - N_{occ}| - \sigma_{n-1}\epsilon)$;
  $W_n = \sigma_n I$, $B_n = (I - W_n)S_{n-1}$;
  $S_n = W_nX_n + B_n$;
  $N_s = \sigma_n N_x + (1 - \sigma_n)N_s$; Updated Occupation
  if $IdErr_n <= 0$ **then**
    Converged = true;
  else if $n > 2$ *and* $\sigma_{n-1} \neq \sigma_{n-2}$ *and* $IdErr_n > 4.5 \times (IdErr_{n-2})^2$ **then**
    Converged = true;
end

In this paper, I first discuss the related work done to implement this algorithm on GPUs. Then I discuss the methodology for the C++ implementation of parallelizing this Deep NN SP2 algorithm (Algorithm 1) on GPUs. Then I evaluate the performance of my implementation and compare it with the current state of the art.

II. Related Work

For accelerating this algorithm on GPUs paper [1] discusses the mixed-precision strategy and use of tensor cores, and has an initial implementation of the above method on GPUs using python. An explicit description of the implementation procedure is not described in [1] and hence it is not clear about the operations to be done on CPUs and GPUs to get maximum throughput performance.

III. Methodology

A flowchart along with the algorithmic steps involved in my C++, CUDA implementation is shown in fig. 2.

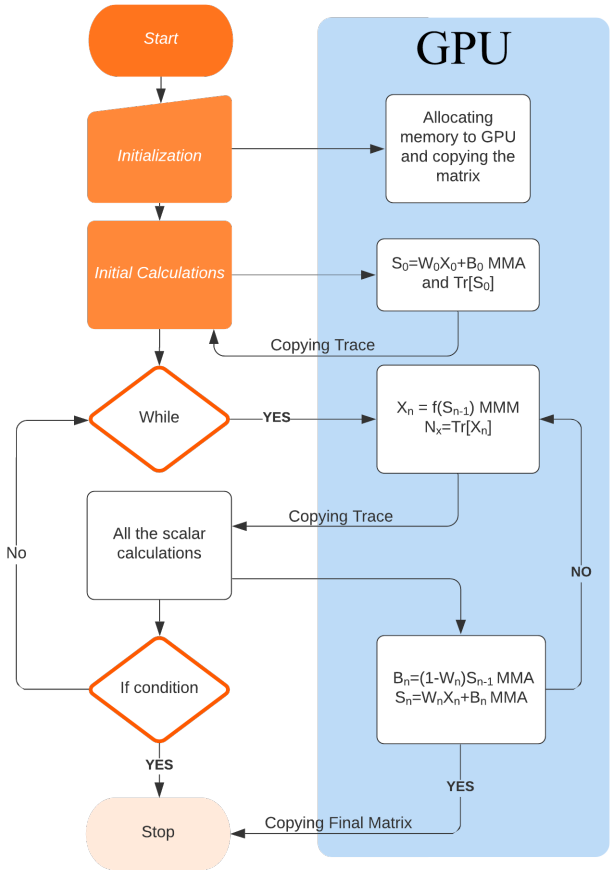


Fig. 2. Flow Chart of the implementation of the Deep NN Sp2 recursive Fermi-operator expansion, denoting the calling of different functions in the algorithm 1 and copying of data between CPU and GPU

The computations done on the GPU were calculating the activation function (matrix-matrix multiplication), matrix-matrix addition, and calculation of trace.

A. Calculation of Trace

For calculation of trace, I used two kernels 1) Calculates the sum of block size number of diagonal elements of the matrix and store it to a new array to the index equal to the block index. Here, the memory access is not a coalesced memory access as diagonal elements are not stored in contiguous memory addresses. These elements are first stored in shared memory, and the reduction operation, as shown in fig. 3, is used to calculate their sum. 2) Calculates the sum of elements of the new array formed by kernel one if the size of the matrix is larger than the block size of the kernel one. Here the memory

access will have coalesced access as these elements are stored in contiguous memory. And the sum will be calculated in the same way as the kernel one.

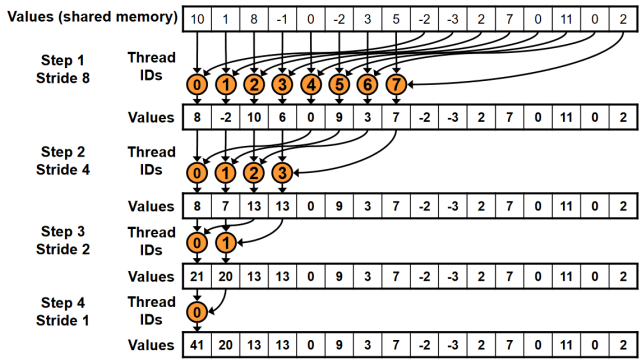


Fig. 3. Reduction operation for calculation of sum of all the array elements

B. Calculation of matrix-matrix Addition and Multiplication

I used the cuBLAS library of Nvidia, a GPU accelerated library, for these calculations. This library is best optimized for square matrix-matrix multiplication. The functions which I used are :

1) For matrix-matrix addition:

cublasSgeam(cublasHandle_t handle, cublasOperation_t transa, cublasOperation_t transb, int m, int n, const float *alpha, const float *A, int lda, const float *beta, const float *B, int ldb, float *C, int ldc);

2) For matrix-matrix multiplication:

cublasSgemm(cublasHandle_t handle, cublasOperation_t transa, cublasOperation_t transb, int m, int n, int k, const float *alpha, const float *A, int lda, const float *B, int ldb, const float *beta, float *C, int ldc)

The cublas handle, which is one of the arguments of these functions, is very important to put some conditions on all the functions performed using this handle. For example, if we have to perform some function on a particular stream, we set the handle using cublasSetStream() to that stream and use this handle to perform the function. Similarly, we can apply other conditions like changing the math mode, changing the pointer mode, etc., using similar functions like cublasSetStream().

C. Calculation of matrix-matrix using Tensor cores

To perform the mixed-precision strategy to do matrix-matrix multiplication, I used Tensor cores available in V100 GPUs. They can execute the mixed-precision strategy for FP32 systems. To use the tensor cores, we set the math mode of the handle to CUBLAS_TENSOR_OP_MATH, which uses tensor cores for its calculations wherever possible. If the math mode is set to CUBLAS_PEDANTIC_MATH, the function strictly follows the normal evaluation mode.

The function used to set math mode to a handle is:
cublasSetMathMode(handle, CUBLAS_TENSOR_OP_MATH);

The convergence condition is checked on the CPU. It requires the trace value of X_n, that is copied from the GPU in each iteration.

IV. Experiments and Results

A. Experiment Setup

For this experiments, the Hamiltonian matrix was generated in the code itself with the same approach used by [1]. The time needed for the GPU implementation on C++ without using tensor cores and with using tensor cores were computed using proper math mode for the handle. To calculate the time required for sequential code, I used the python code provided by [1] in their supporting materials. Then first, the speed up with respect to the sequential code was computed. Then the performance in terms of teraFLOPS was calculated for my implementation and compared with the performance of the paper [1] version.

B. Results

Speedups with respect to the sequential algorithm are shown in fig. 4. As can be observed, we got a considerable speedup, and speedup increased with an increase in the matrix size. This speedup will increase until a specific limit corresponding to the memory constraint of the GPU. Also, when the tensor cores were used, the speedups were further improved and reached almost 750x for the 2000*2000 matrix.

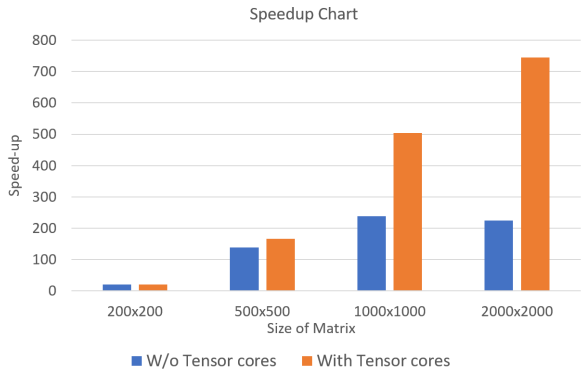
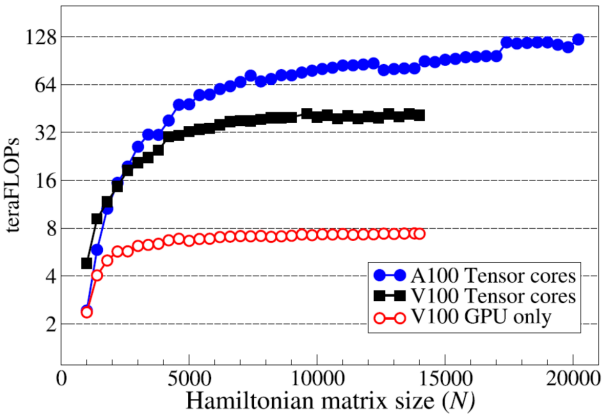


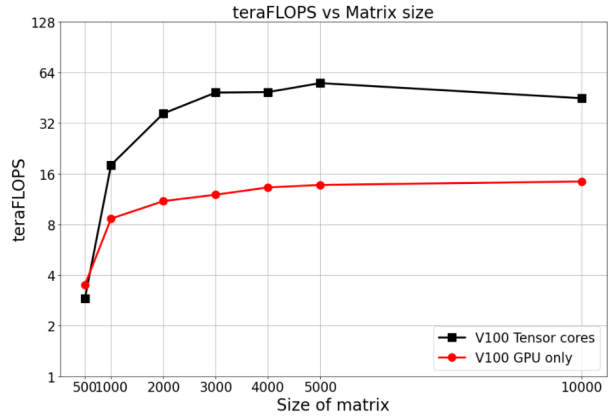
Fig. 4. Speedups Graph

In fig. 5 two plots are shown, (a) which was the performance plot given by [1] (b) was the performance plot for my implementation. As can be observed, My implementation was faster in most cases than the existing implementation in the paper [1] when compared for V100 GPUs. It was almost reaching the performance of A100 GPUs. Both with and without tensor cores implementations were comparatively faster.

The possible reason for getting this speedup is that I used the best possible libraries for matrix operations. Also, trace calculations for a bigger matrix were done such that coalesced memory access can be used.



(a)



(b)

Fig. 5. (a) Performance plot of implementation of [1] (b) Performance plot for C++ (CUDA) implementation

V. Conclusions

The main bottleneck in the algorithm was the activation function, i.e., the squaring of the matrix. My C++ implementation was faster in most cases than the existing implementation in the paper [1] when compared on V100 GPUs. The mixed precision approach gave good speedups compared to the normal matrix-matrix multiplication.

To further extend this work we can make use of multi-GPU architecture for large matrices. Further as the matrix-matrix multiplication computation becomes faster (as it is a constantly evolving problem in parallel programming), this algorithm can achieve higher performances.

References

[1] J. Finkelstein, J. S. Smith, S. M. Mniszewski, K. Barros, C. F. A. Negre, E. H. Rubensson, and A. M. N. Niklasson. Mixed precision fermi-operator expansion on tensor cores from a machine learning perspective. *Journal of Chemical Theory and Computation*, 17(4):2256–2265, 2021. PMID: 33797253.